

COMP 424 - Artificial Intelligence

Alpha-Beta Game Search

Readings: R&N Ch 5

Lecture 7: September 23, 2026

Jackie CK Cheung (jackie.cheung@mcgill.ca)

Su Lin Blodgett (sulin.Blodgett@mcgill.ca)

Quick recap

Minimax method extended brute force search to the game playing domain:

- What are the key assumptions of Minimax search?
- What is the opponent model assumed by Minimax search?

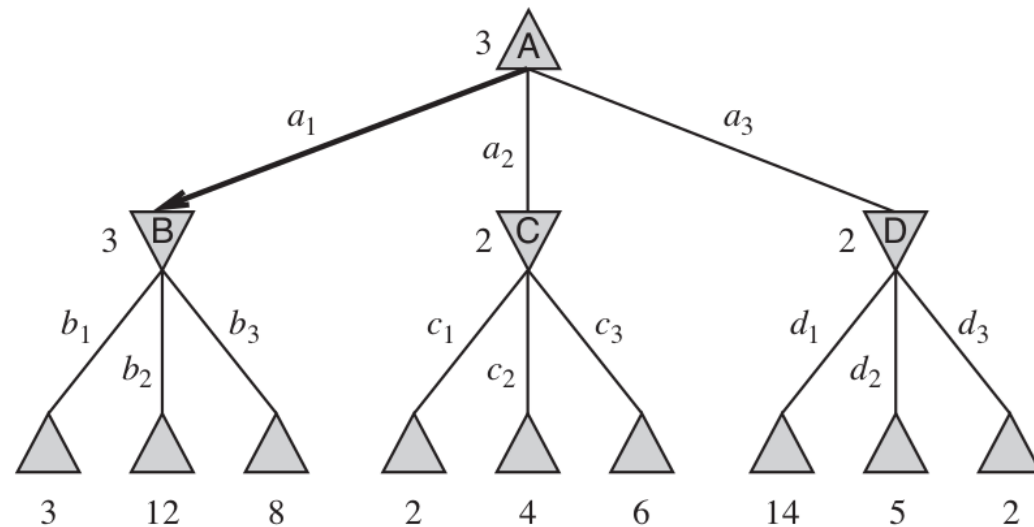
Recap: A Simple Game Tree

MINIMAX(Node n) =

$$\begin{cases} \text{UTILITY}(n) & \text{if } n \text{ is a terminal state} \\ \max_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MAX node} \\ \min_{s \in \text{Succ}(n)} \text{MINIMAX}(s) & \text{if } n \text{ is a MIN node} \end{cases}$$

MAX

MIN



Speeding up minimax

Can we leverage problem structure to save time?

Key insight: stop searching in the current direction if we know that we won't reach this part of the game tree.

- This can happen if the current state of the game is too favourable for one or the other player, compared to the parts of the game tree that we have already visited!

Formalized with the alpha-beta pruning method. A real key tool in chess up to today, heavily used in Deep Blue!

α - β pruning

- **Simple idea:** keep track of “range” of outcomes from past search, **prune** non-competitive moves for both players
 - Max/min structure **limits each players' gain**
 - After some branches complete, next branches that are too good for one or the other player will never be selected. Stop early!
- α - β is a standard technique for deterministic, perfect information games
- How does it work?
 - It uses two parameters, α and β , to track bounds on the utility or evaluation values.
 - **MAIN IDEA:** True outcome must lie between α and β

α - β pruning

- Proceed like Minimax, with DFS and then back ups
- Additionally, keep track of:
 - the best (**highest**) leaf value found for **Max** (in α)
 - the best (**lowest**) leaf value found for **Min** (in β)
 - Call every node with the range guaranteed by previous search

α - β pruning

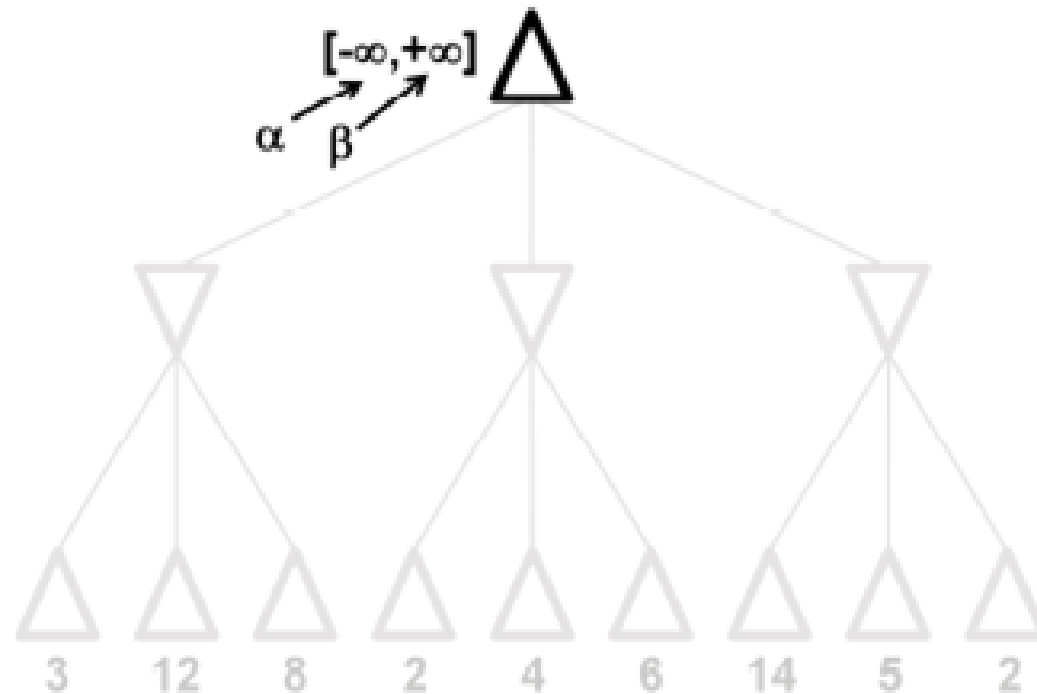
- We are surely allowed to prune in the event of inconsistency ($\alpha \geq \beta$), because:
 - We just made an update to α or β indicating a branch exists for us that is “too good”. It lies outside the existing range.
 - If we returned this value through back-up, it will never be selected by the parent in the min/max interaction
 - As the “actor” is performing $\max(\alpha)$ or $\min(\beta)$, continuing to search more options here can never fix the inconsistency
- Therefore, we stop looping over our options. Return our α / β safely without further computation. Our value is not used, but time is saved!

α - β pruning algorithm

```
double MaxValue( $s, \alpha, \beta$ )
    if cutoff( $s$ ), return Evaluation( $s$ ).
    for each state  $s'$  in Successors( $s$ )
        let  $\alpha = \max \{ \alpha, \text{MinValue}(s', \alpha, \beta) \}$ .
        if  $\alpha \geq \beta$ , return  $\beta$ .
    return  $\alpha$ .

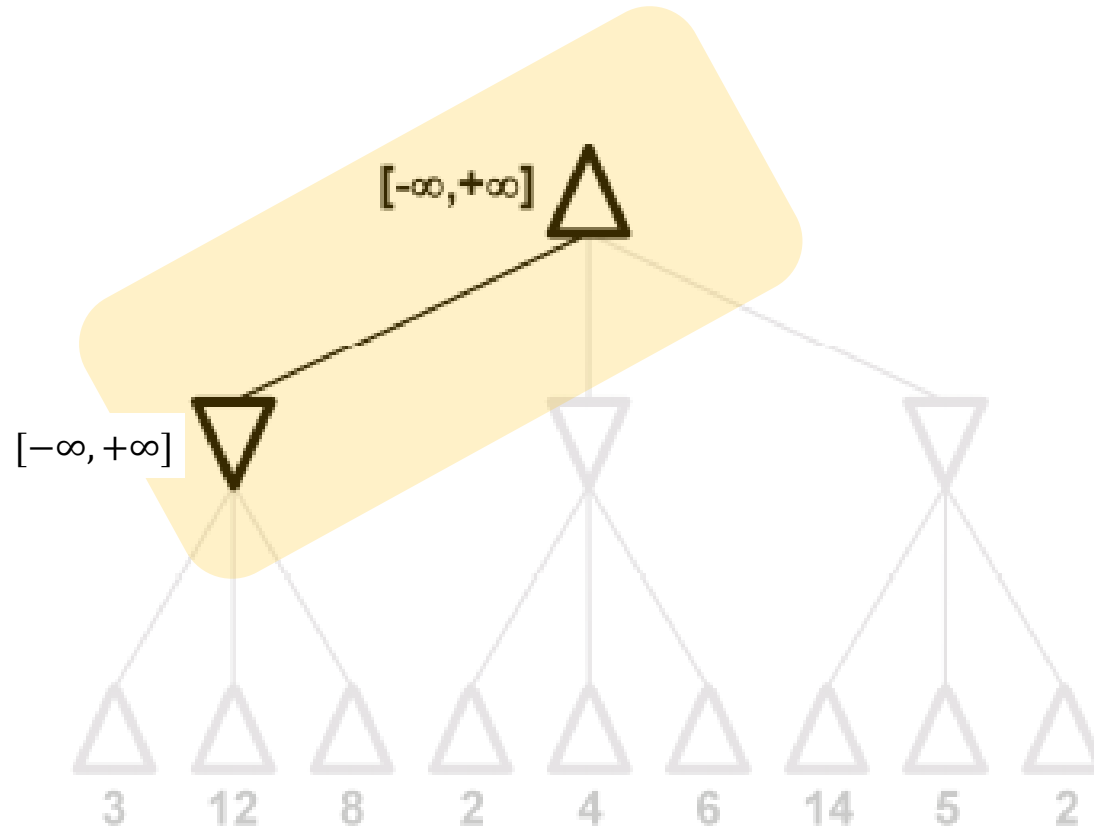
double MinValue( $s, \alpha, \beta$ )
    if cutoff( $s$ ), return Evaluation( $s$ ).
    for each state  $s'$  in Successors( $s$ )
        let  $\beta = \min \{ \beta, \text{MaxValue}(s', \alpha, \beta) \}$ .
        if  $\alpha \geq \beta$ , return  $\alpha$ .
    return  $\beta$ .
```

Example



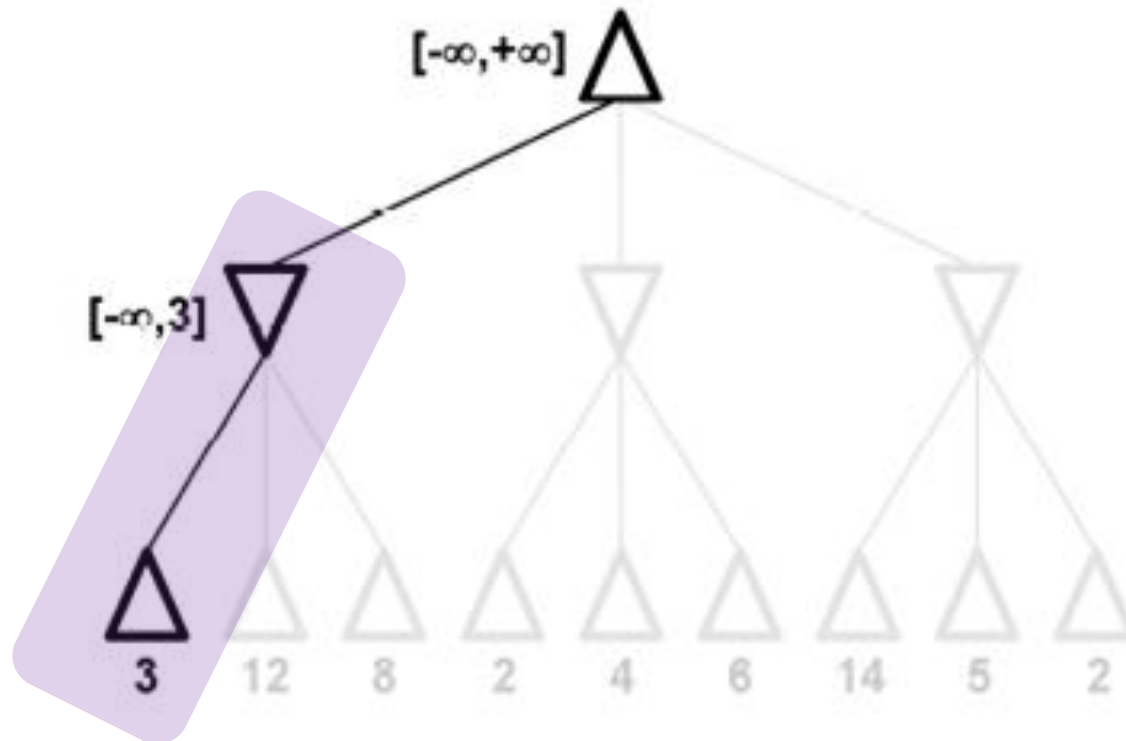
Initialize α and β

Example



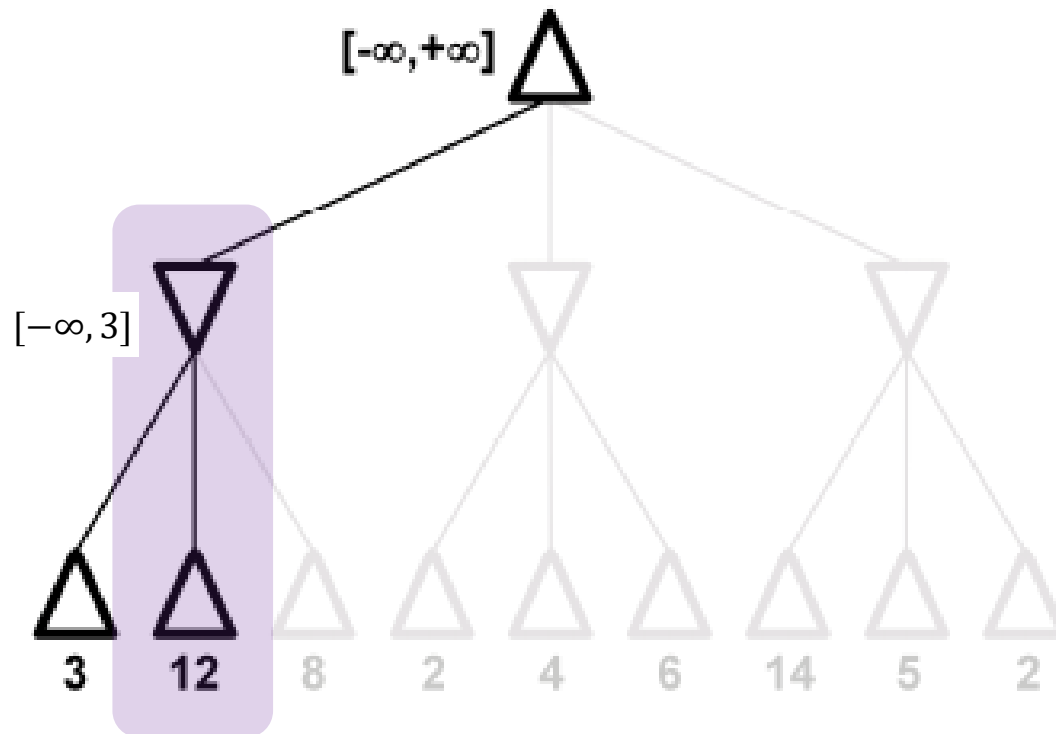
Search the first move for Max
(trying to update α)

Example



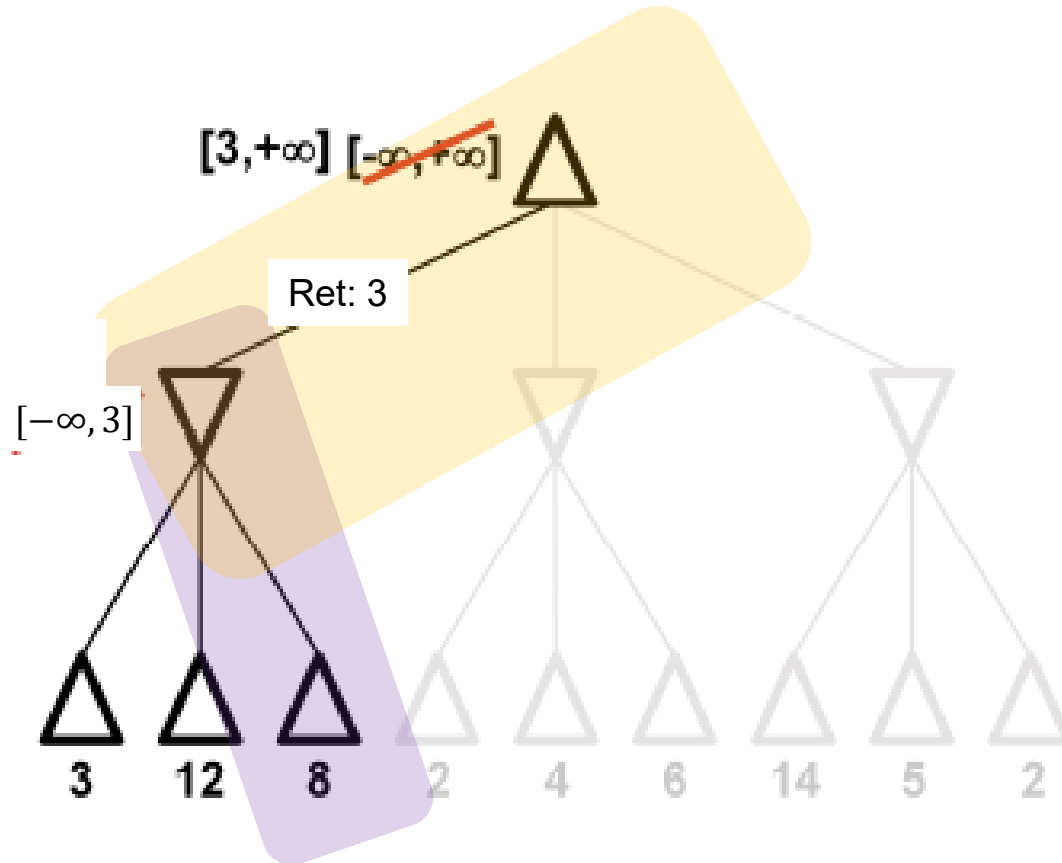
Search the first move for Min
(update β to 3)

Example



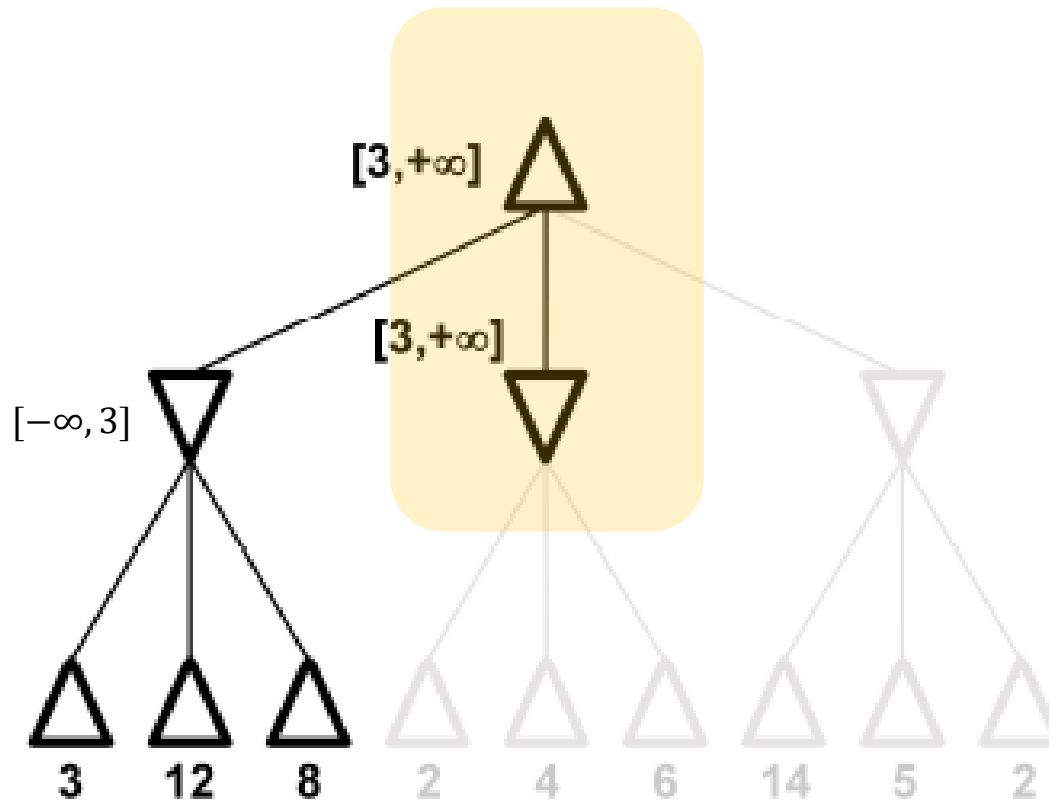
Consider the second move for Min: not better than 3!
(no change to β)

Example



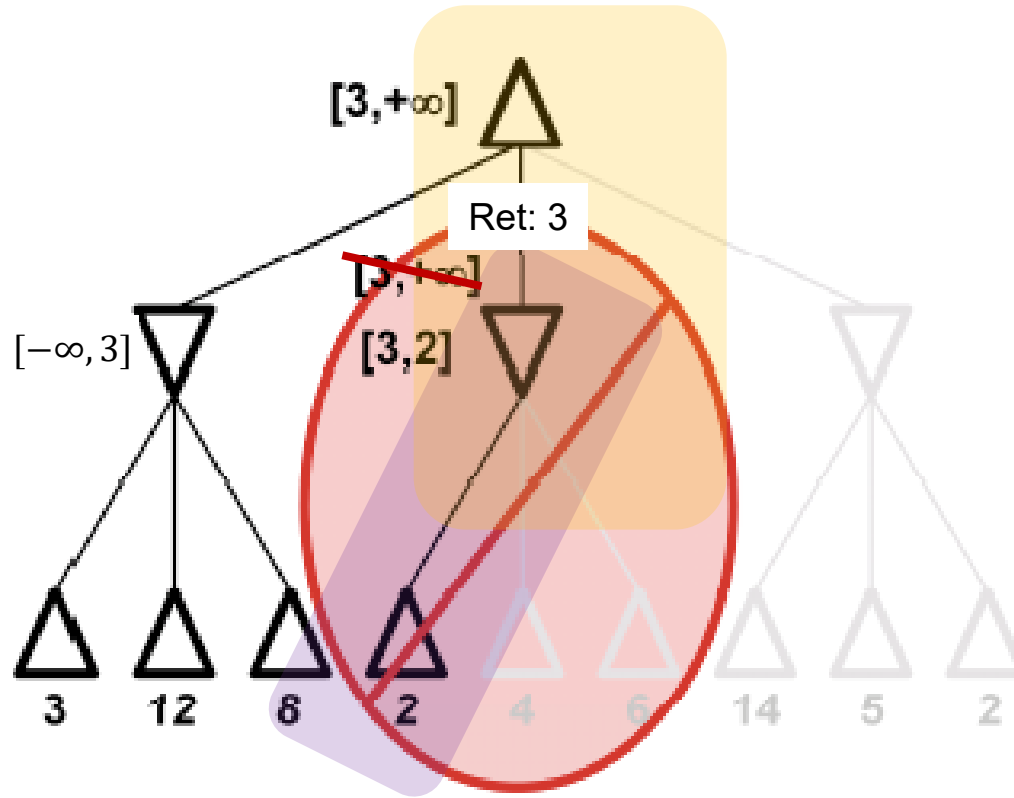
Consider the third move for Min: not better than 3.
Return 3 from min node (write $[3, 3]$).
Update α at top level for Max.

Example



Consider second move for Max, passing in $[3, +\infty]$

Example

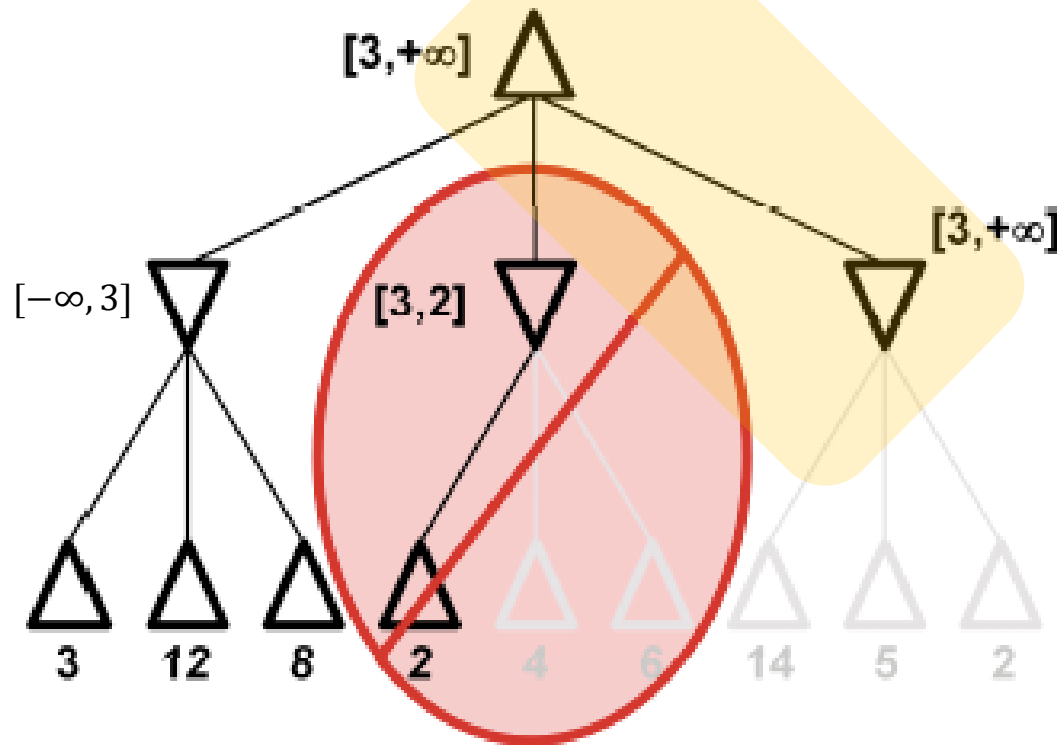


Consider first move of Min

Update β and notice inconsistency $[3, 2]$

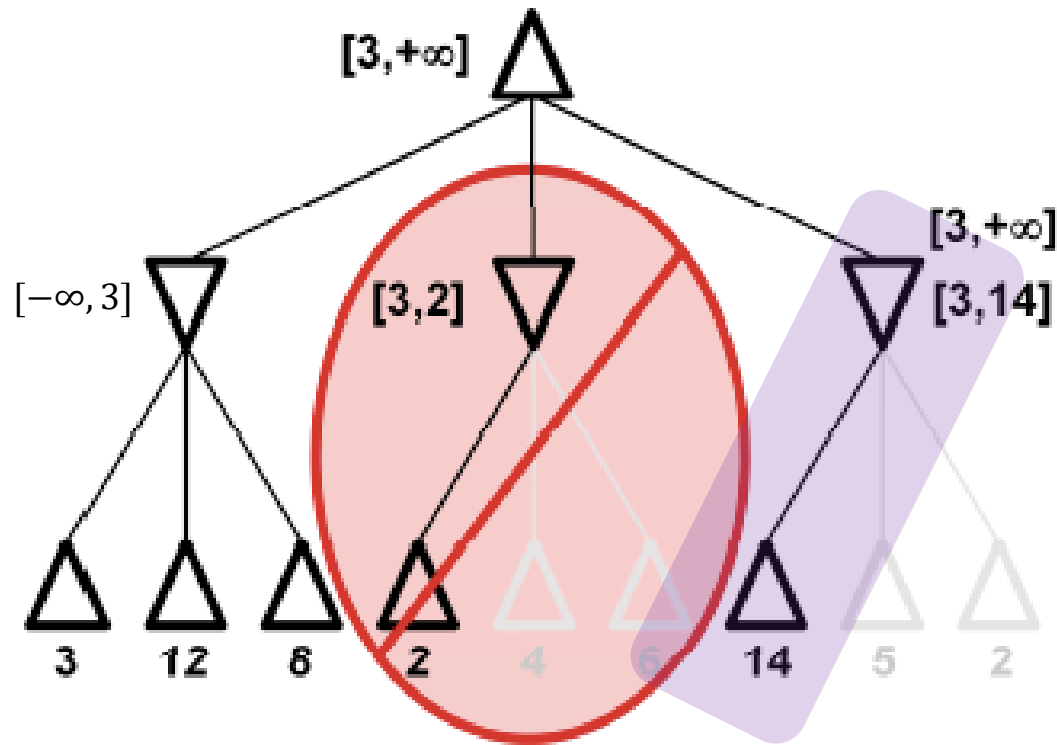
Return 2 to top **immediately**, (no change to α , as 3 is better)

Example



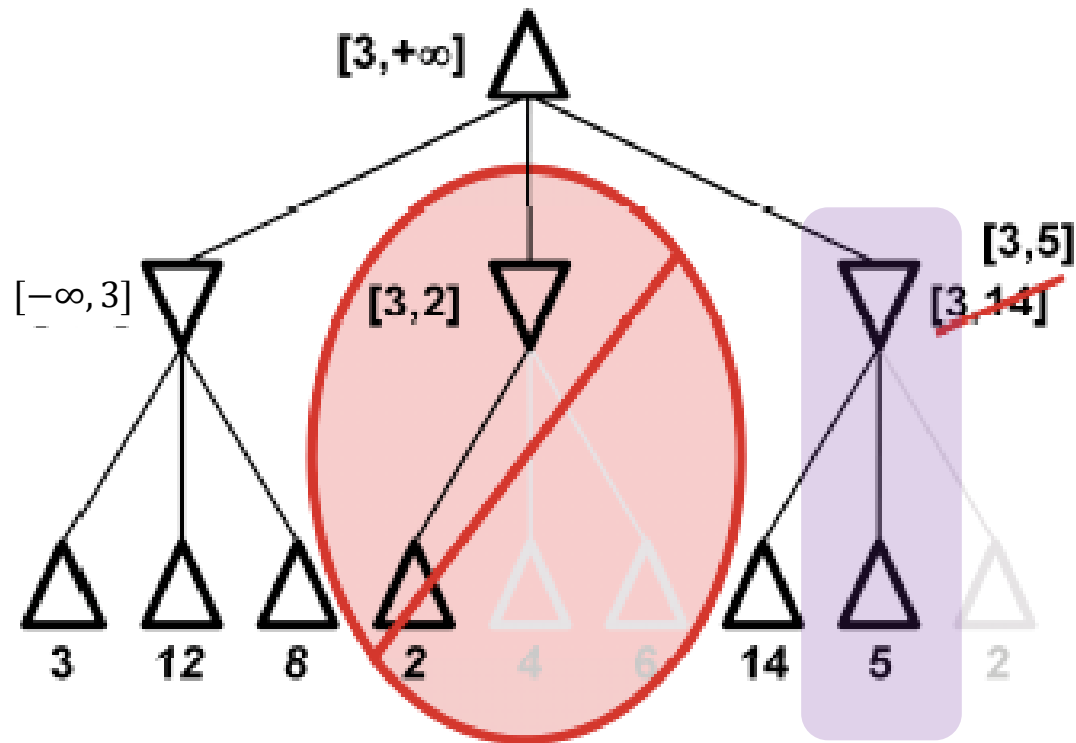
Consider third move of Max, passing in $[3, +\infty]$

Example



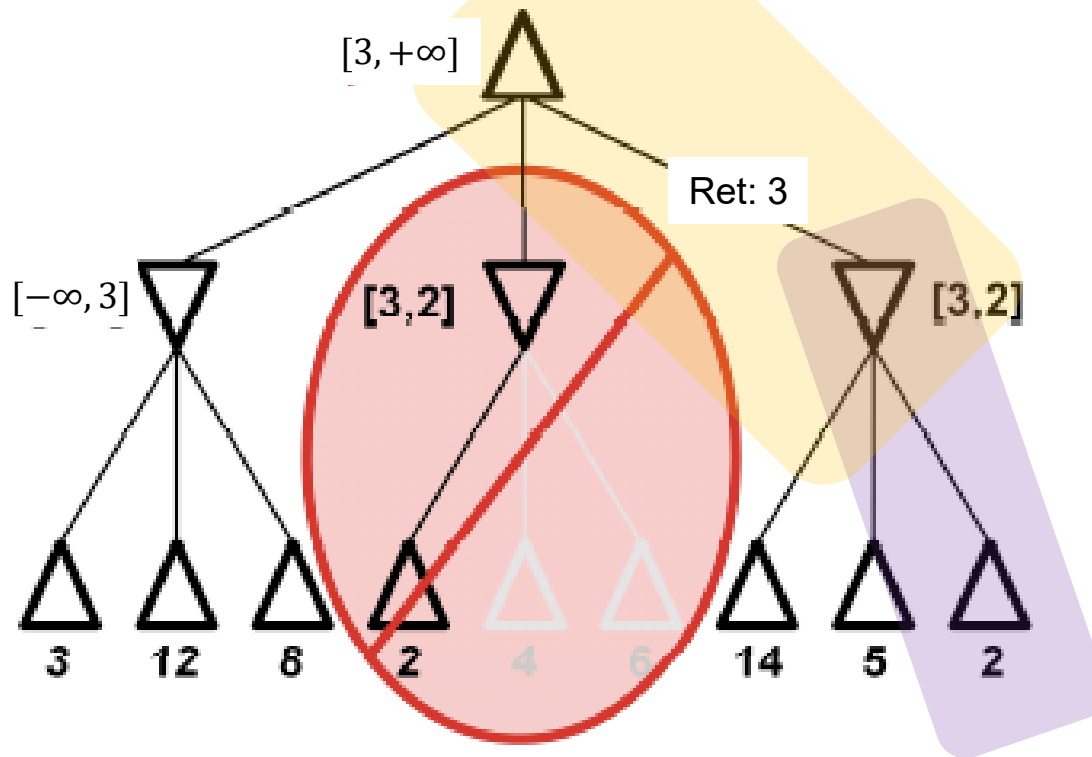
Consider first move of Min
Update β to 14

Example



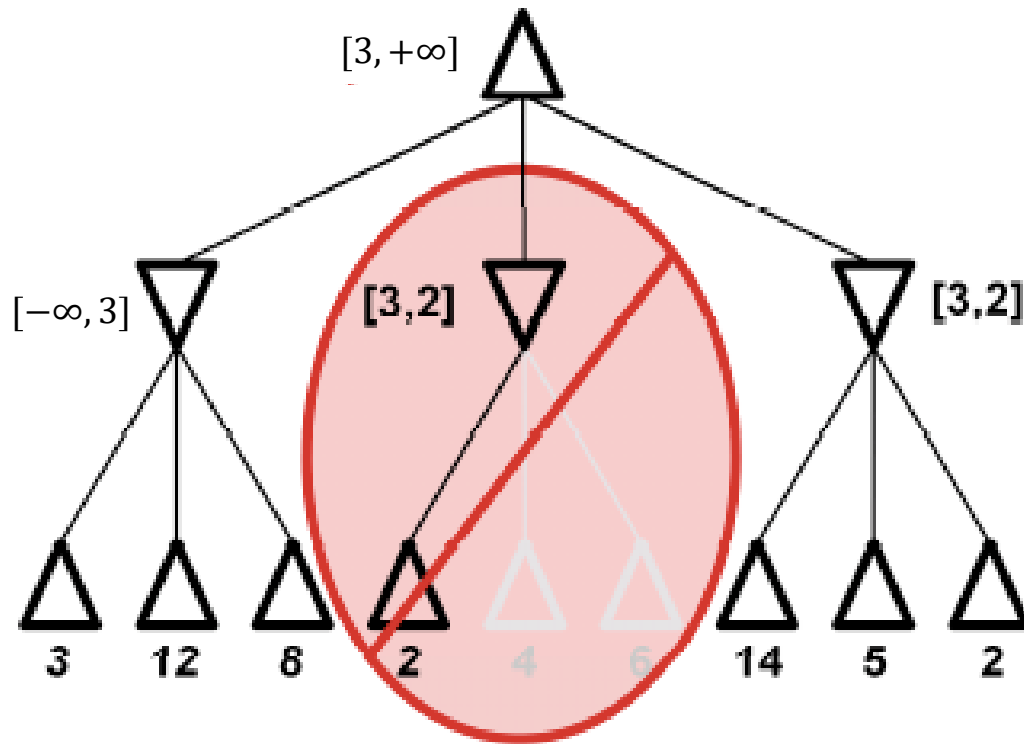
Consider second move of Min
Update β to 5

Example



Third move: update β and notice inconsistency $[3, 2]$
Return 2 to top, but this time it doesn't save us any effort
Return 3 at the top level (done!)

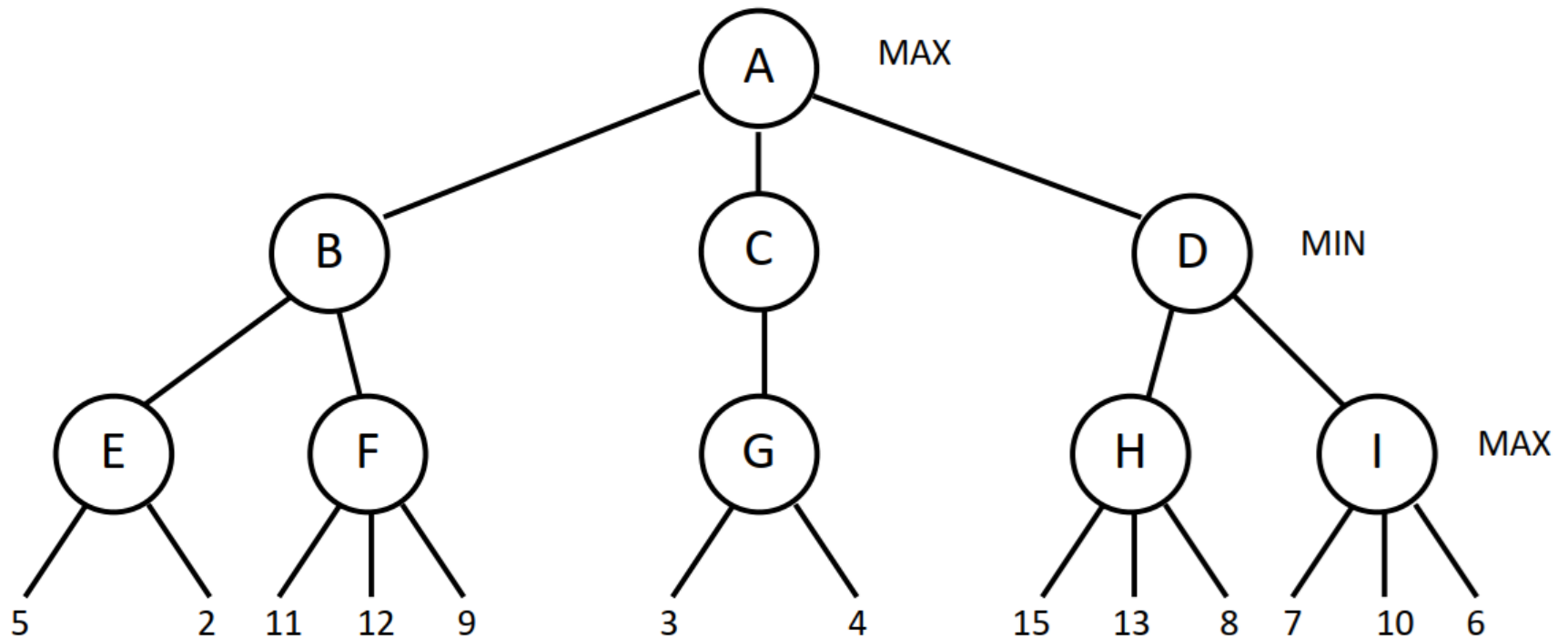
Example



In this search, we pruned away two leaf nodes compared to Minimax. Overall value is 3 with play proceeding on left-most branch.

Your Turn

- Run alpha-beta pruning on this game search tree



Note

- The textbook's version of this example (Edition 3, p. 168, Figure 5.5) shows the correct steps, but with incorrect and confusing alpha and beta values, which don't match their algorithm!
 - There is the abstract idea of upper and lower bounds, and they mix this logic with the variable assignments to alpha and beta.

Analyzing α - β pruning's efficiency

- It depends strongly on move order, as this affects what the α - β values are when we hit at each decision point
 - Globally best moves first, then worse moves immediately whenever pruning is possible maximizes pruning.
- General rules:
 - We can never prune only the left-most branch of a sub-tree
 - When we prune a parent, all children are pruned
 - When a branch indicates pruning, all further-right branches are pruned

Analyzing α - β pruning's efficiency

- It depends strongly on move order, as this affects what the α - β values are when we hit at each decision point
 - Globally best moves first, then worse moves immediately whenever pruning is possible maximizes pruning.
- Pattern is a bit complicated, but amortized analysis yields:
 - Run-time $b \times 1 \times b \times 1 \times \dots \approx O(b^{\frac{d}{2}}) = \sqrt{O(b^d)}$.
- Worst-case runtime is still $O(b^d)$
 - Worst move order means we keep finding ways to improve until the bottom-right of the tree, we never prune anything.
 - Same as minimax search, with some trivial book-keeping.

Discussing α - β pruning gains

- Summary of outcomes:
 - With bad move ordering, time complexity is $O(b^m)$
 - With perfect ordering, time complexity is $O(b^{m/2})$
 - On average, $O(b^{3m/4})$, if we expect to find the max/min after $b/2$ expansions
 - Randomizing the move ordering can achieve the average
- Overall, α - β leads to about doubling the search depth for a given amount of computation.
 - Compare to doubling a minimax search depth by spending more run-time or buying stronger CPU – exponential investment
 - This is a fundamental breakthrough in AI gameplay!

Important lessons of α - β pruning

- Pruning does not affect the final result.
 - Value estimated and best moves are same as returned by Minimax (both are optimal in minimax view)
- α - β pruning demonstrates the value of **reasoning about which computations are important!**
- Can be paired well with game knowledge, solutions of weaker solvers, human game analysis, deep learning
 - Getting the “right” α - β values cheaply from these sources will let us prune with less preliminary searching

Drawbacks of α - β

- If the branching factor is really big, search depth is still too limited. E.g., Go, where branching factor is about 300
- When mixed with evaluation functions (approximate), our view on the range is also approximate.

Forward pruning

Idea (for domains with large branching factor):

Only explore the n best moves for current state
(according to the evaluation function)

- Unlike α - β pruning, this can lead to suboptimal solutions
- But it can be very efficient with a good evaluation function

State-of-the-art game playing programs

Chinook (Schaeffer et al., U. of Alberta)

- Best checkers player (since 1990's)
- Plain α - β search, performed on standard PCs
- Evaluation function based on expert features of the board
- Opening move database
- HUGE endgame database!
Chinook has perfect information for all checkers positions involving 8 or fewer pieces on the board (a total of 443,748,401,247 positions)
- Only a few moves in middle of the game are actually searched
- They've now done an exhaustive search for checkers, and through optimal play can force at least a draw

Deep Blue (IBM)

- Specialized chess processor, special-purpose memory architecture
- Very sophisticated evaluation function (expert features, tuned weights)
- Database of standard openings/closings
- Uses a version of α - β pruning (with undisclosed improvements)
 - Can search up to 40-deep in some branches
- Can search over 200 billion positions per second!
- Overall, an impressive engineering feat

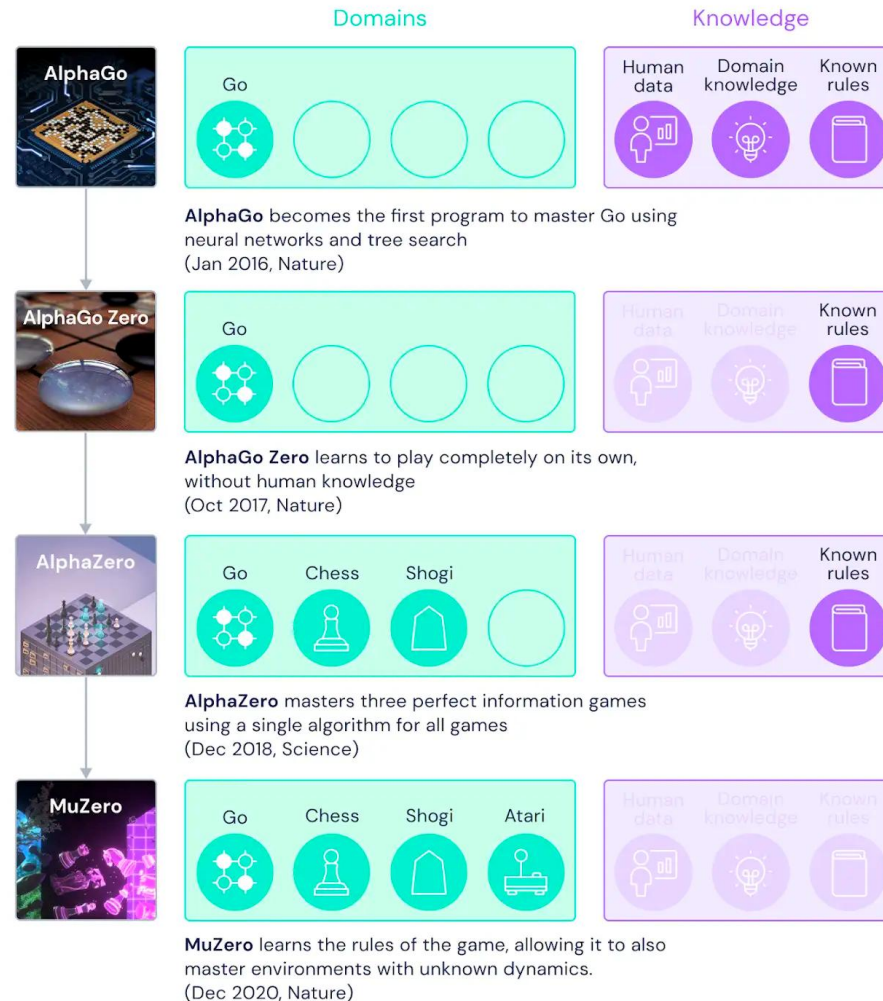
Computer Chess

- Now, several computer programs running on regular hardware are on par with human champions (e.g., Fritz, Stockfish)
- ELO ratings of top chess bots are estimated at over 3500
 - C.f., rating of 2500 needed to become a Grandmaster
 - Top human players at just over 2800
- Human chess players use chess bots as a way to analyze and improve their game
 - Interesting preview of how humans and AI can cooperate?

AlphaGo (DeepMind, 2016)

- Uses **Monte Carlo tree search** (more on this next class!) to simulate future game states
- Uses **deep reinforcement learning** (i.e., **multi-layer neural networks + reinforcement learning**) to learn how to value moves (**policy network**) and board states (**value network**):
 - These are machine learning techniques
 - Requires access to a database of previous games by expert players
- Performance:
 - **March 2016**: 4W-1L record against Lee Sedol
 - **2017**: 60W-0L record after further training against human pros
- AlphaZero (2017) removed the need for human game data, learned to play “from scratch” against itself!

MuZero (DeepMind, 2020)



Some possible strategies for AI in games

- Design a compact state representation
- Search using iterative deepening for real-time play
- Use alpha-beta pruning with an evaluation function
 - Order moves using the evaluation function
 - Tune the evaluation function using domain knowledge, trial-and-error, learning
 - Searching deeper is often more important than having a good evaluation function
- Consider using different strategies for opening, middle and endgame (including look-ups)
- Consider that the opponent might not be optimal
- Decide where to spend the computation effort!

Summary

- Understand the different types of games
- Understand Minimax and alpha-beta pruning, what they compute, how to implement, and common methods to make them work better (e.g., node ordering)
- Understand the role of the evaluation function and desirable characteristics
- Get familiar with the state-of-the-art for solving some common games

Extra Reading (R&N)

- Stochastic games
 - Add chance nodes to the search tree,
 - Minimax $\rightarrow$ Expectiminimax
 - α - β applies (with modifications for chance nodes)
- More sophisticated move ordering schemes
 - killer moves
 - transposition tables
- More sophisticated cutoffs
 - quiescence search
 - singular extensions
- Multiplayer games

Human or computer - who is better?

Checkers:

- 1994: Chinook (U.of A.) beat world champion Marion Tinsley, ending 40-yr reign.

Othello:

- 1997: Logistello (NEC research) beat the human world champion.
- Today: world champions refuse to play AI computer program (because it's too good).

Chess:

- 1997: Deep Blue (IBM) beat world champion Gary Kasparov.
- 2002: Fritz drew with world champion Vladimir Kramnik.

Backgammon:

- 1992: TD-Gammon (IBM) is world champion amongst humans and computers

Go:

- 2016: AlphaGo (Google) beats top-ranked players Lee Sedol and Ke Jie

Human or computer - who is better?

Scrabble:

- 1998: Maven (UofA) beats world champion Adam Logan 9-5.
- Knowing the whole dictionary helps a lot!

Bridge:

- 1988: Ginsberg's program places 12th in world championships.
- Coordination with partner is still very difficult.

Poker:

- 2015: UofAlberta group announces *Heads-up limit Texas hold 'em* is solved.
- Other variants of the game are still open; annual competition of AI bots.
- Still very difficult to adapt to changing opponents.

Commercial, multi-player games:

- Very hard problems, progress slowly being made.
- Real-time, opponents change, dynamic, cannot see everything...
- Goal is often not to beat human players, but to provide “interesting” opponents.